1. Business Context

This project is based on a marketing case study for a mobile language learning app operating on a monthly subscription model.

Before moving to KPI reconstruction and dashboard development, the dataset must first be profiled and cleaned to ensure that the final analysis is based on reliable and internally consistent data.

The objective of this stage is to inspect the raw dataset, identify structural and logical data quality issues, correct recoverable inconsistencies, remove corrupted records, and prepare a clean analytical table for the next stage of the project.

In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

2. Data Loading and Initial Inspection

Load the raw dataset into a pandas DataFrame. low_memory=False is used to avoid mixed-type inference warnings during import.

In [2]:
```
df = pd.read_csv(
    "C:/Users/glazk/Downloads/marketing_raw_2025_case_ready.csv",
    low_memory=False)
df.head() # Verify that the file has been loaded
```

Out[2]:

	user_id	event_date	platform	country	creative	impressions	clicks	install_fla
0	1	2025-05-01 10:51:00	iOS	USA	Video_UGC_01	773	16	
1	2	2025-05-06 17:48:00	iOS	USA	Video_UGC_01	75	1	
2	3	2025-05-05 06:01:00	iOS	USA	Video_UGC_01	1129	14	
3	4	2025-05-04 01:42:00	iOS	USA	Video_UGC_01	76	3	
4	5	2025-05-04 02:05:00	iOS	USA	Video_UGC_01	1715	32	

In [3]:
```
df.info()  # Review the dataset structure, column types, and memory usage.
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 25000 entries, 0 to 24999
Data columns (total 11 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   user_id            25000 non-null  int64
 1   event_date         24970 non-null  object
 2   platform           23991 non-null  object
 3   country            24212 non-null  object
 4   creative           24238 non-null  object
 5   impressions        25000 non-null  int64
 6   clicks             25000 non-null  int64
 7   install_flag       25000 non-null  int64
 8   subscription_flag  25000 non-null  int64
 9   revenue_usd        25000 non-null  float64
 10  spend_usd          25000 non-null  float64
dtypes: float64(2), int64(5), object(4)
memory usage: 2.1+ MB
```

In [4]:
```
# Convert the event_date column to datetime format.
# Invalid date values, if any, will be converted to NaT.
df["event_date"] = pd.to_datetime(df["event_date"], errors="coerce")
```

In [5]:
```
df.info() # Revalidate the dataset
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 25000 entries, 0 to 24999
Data columns (total 11 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   user_id            25000 non-null  int64
 1   event_date         24940 non-null  datetime64[ns]
 2   platform           23991 non-null  object
 3   country            24212 non-null  object
 4   creative           24238 non-null  object
 5   impressions        25000 non-null  int64
 6   clicks             25000 non-null  int64
 7   install_flag       25000 non-null  int64
 8   subscription_flag  25000 non-null  int64
 9   revenue_usd        25000 non-null  float64
 10  spend_usd          25000 non-null  float64
dtypes: datetime64[ns](1), float64(2), int64(5), object(3)
memory usage: 2.1+ MB
```

The raw CSV file is loaded into pandas and reviewed through an initial inspection of the first rows, column types, row count, and memory usage.

At this stage, the "event_date" column is converted to datetime format to ensure correct handling in profiling and later time-based analysis.

3. Data Profiling, Quality Assessment and Data Cleaning

Before applying any transformations, the dataset is profiled to identify issues that could distort the analysis.

The profiling focuses on:

- missing values in key descriptive fields,

- exact duplicate rows,
- broken funnel logic,
- impossible ad interaction records,
- and inconsistent category labels.

This step helps distinguish between recoverable issues and rows that are too corrupted to be reliably used in analysis.

After identifying the main data quality issues, the dataset is cleaned in a structured order.

The cleaning process includes:

- removing exact duplicate rows,
- correcting subscription tracking inconsistencies,
- correcting install tracking inconsistencies where possible,
- removing impossible interaction rows,
- standardizing categorical values across platform, country, and creative fields.

The goal is to preserve valid business information while ensuring that the final dataset is analytically consistent.

In [6]:
```
df.isna().sum()
# Check the number of missing values in each column.
# Missing values in key descriptive fields may affect grouping and aggregation l
```

Out[6]:
```
user_id                 0
event_date             60
platform             1009
country               788
creative              762
impressions             0
clicks                  0
install_flag            0
subscription_flag       0
revenue_usd             0
spend_usd               0
dtype: int64
```

The missing value check is used to assess dataset completeness and identify whether critical analytical fields such as date, platform, country, or creative require cleaning before further analysis.

In [7]:
```
df.duplicated().sum() # Check for duplicate rows in the dataset
```

Out[7]:
```
np.int64(800)
```

In [8]:
```
# Inspect duplicate rows before removing them.
# keep=False shows all rows that belong to duplicate groups.
dup_rows = df[df.duplicated(keep=False)].sort_values(
    by=["user_id", "event_date", "platform", "country", "creative"]
)

dup_rows.head(20)
```

Out[8]:

	user_id	event_date	platform	country	creative	impressions	clicks	insta
18	19	2025-05-06 20:30:00	iOS	USA	Video_UGC_01	1995	37	
24868	19	2025-05-06 20:30:00	iOS	USA	Video_UGC_01	1995	37	
29	30	2025-05-04 02:33:00	iOS	USA	Video_UGC_01	1324	27	
24242	30	2025-05-04 02:33:00	iOS	USA	Video_UGC_01	1324	27	
72	73	2025-05-04 22:55:00	iOS	USA	Video_UGC_01	527	12	
24974	73	2025-05-04 22:55:00	iOS	USA	Video_UGC_01	527	12	
76	77	2025-05-03 11:57:00	iOS	USA	Video_UGC_01	52	4	
24387	77	2025-05-03 11:57:00	iOS	USA	Video_UGC_01	52	4	
81	82	2025-05-04 13:14:00	iOS	USA	Video_UGC_01	3151	44	
24975	82	2025-05-04 13:14:00	iOS	USA	Video_UGC_01	3151	44	
83	84	2025-05-06 20:14:00	iOS	USA	Video_UGC_01	1247	17	
24928	84	2025-05-06 20:14:00	iOS	USA	Video_UGC_01	1247	17	
85	86	2025-05-05 06:44:00	iOS	USA	Video_UGC_01	1491	36	
24338	86	2025-05-05 06:44:00	iOS	USA	Video_UGC_01	1491	36	
90	91	2025-05-04 21:52:00	iOS	USA	Video_UGC_01	348	6	
24474	91	2025-05-04 21:52:00	iOS	USA	Video_UGC_01	348	6	
92	93	2025-05-06 03:28:00	iOS	USA	Video_UGC_01	608	9	
24409	93	2025-05-06 03:28:00	iOS	USA	Video_UGC_01	608	9	
99	100	2025-05-01 02:18:00	iOS	USA	Video_UGC_01	1451	22	
24862	100	2025-05-01 02:18:00	iOS	USA	Video_UGC_01	1451	22	

Duplicate rows are reviewed before removal to confirm that they are exact row-level duplicates rather than legitimate repeated user events. Keeping such rows would inflate campaign totals and distort the analysis.

In [9]:
```
df = df.drop_duplicates()
```

Check for rows where revenue exists but no subscription is recorded. This violates business logic because subscription revenue should not exist without a subscription event.

In [10]:
```
df[(df["revenue_usd"] > 0) & (df["subscription_flag"] == 0)] # Revenue without s
```

Out[10]:

	user_id	event_date	platform	country	creative	impressions	clicks	ins
44	45	2025-05-01 02:44:00	iOS	USA	Video_UGC_01	2242	49	
49	50	2025-05-06 07:47:00	iOS	USA	Video_UGC_01	1682	23	
58	59	2025-05-06 00:27:00	iOS	USA	Video_UGC_01	1225	19	
60	61	2025-05-07 23:20:00	iOS	USA	Video_UGC_01	282	4	
89	90	2025-05-06 00:06:00	iOS	USA	Video_UGC_01	241	7	
...	...	...	...	...	...	...	...	
20711	20712	2025-05-21 05:56:00	Android	Other	Static_Banner_05	332	1	
20807	20808	2025-05-15 21:23:00	Android	Other	Static_Banner_05	107	1	
20824	20825	2025-05-15 06:54:00	Android	Other	Static_Banner_05	866	3	
21145	21146	2025-05-26 20:25:00	iOS	Other	Static_Banner_05	752	4	
21190	21191	2025-05-25 09:14:00	Android	Other	Static_Banner_05	1533	7	

400 rows × 11 columns

Rows with revenue but no recorded subscription indicate a likely tracking inconsistency. Since the observed revenue corresponds to the subscription price, these cases can be treated as recoverable rather than discarded.

In [11]:
```
df.loc[(df["revenue_usd"] == 29.99) & (df["subscription_flag"] == 0), "subscript
```

Found rows with revenue but no subscription. Since revenue matches subscription price ($29.99), this likely indicates tracking issue. Subscription flag was corrected instead of

dropping data.

In [12]:
```
df[(df["subscription_flag"] == 1) & (df["install_flag"] == 0)] # Subscription wi
```

Out[12]:

	user_id	event_date	platform	country	creative	impressions	clicks	ins
21	22	2025-05-03 11:57:00	iOS	USA	Video_UGC_01	2040	26	
93	94	2025-05-03 07:35:00	iOS	USA	Video_UGC_01	2916	55	
116	117	2025-05-01 13:11:00	iOS	USA	Video_UGC_01	463	8	
122	123	2025-05-05 19:30:00	iOS	USA	Video_UGC_01	1488	28	
166	167	2025-05-03 06:14:00	iOS	USA	Video_UGC_01	389	11	
...	...	...	...	...	...	...	...	
20894	20895	2025-05-17 21:37:00	Android	Other	Static_Banner_05	548	6	
20906	20907	2025-05-20 09:38:00	Android	Other	Static_Banner_05	491	2	
20966	20967	2025-05-25 20:09:00	Android	Other	Static_Banner_05	619	4	
21016	21017	2025-05-23 18:59:00	Android	Other	Static_Banner_05	686	2	
21096	21097	2025-05-27 22:11:00	Android	Other	Static_Banner_05	744	4	

300 rows × 11 columns

In [13]:
```
# Inspect revenue pattern for invalid subscription rows
df.loc[(df["subscription_flag"] == 1) & (df["install_flag"] == 0), "revenue_usd"
```

Out[13]:
```
revenue_usd
29.99    300
Name: count, dtype: int64
```

In [14]:
```
# Check whether invalid rows are concentrated on a specific platform
df.loc[(df["subscription_flag"] == 1) & (df["install_flag"] == 0), "platform"].v
```

Out[14]:
```
platform
iOS         221
Android      76
 iOS          1
IOS           1
ANDROID       1
Name: count, dtype: int64
```

In [15]:
```
# Check whether invalid rows are concentrated in specific geos
```

```
df.loc[(df["subscription_flag"] == 1) & (df["install_flag"] == 0), "country"].va
```

Out[15]:

```
country
Norway            172
USA                58
Other              42
United Kingdom     16
New Zealand         5
norway              2
Norway              2
Other               2
Usa                 1
Name: count, dtype: int64
```

In [16]:

```
# Fix invalid funnel rows: if subscription exists with subscription revenue, set
df.loc[
    (df["revenue_usd"] == 29.99) &
    (df["subscription_flag"] == 1) &
    (df["install_flag"] == 0),
    "install_flag"
] = 1
```

In [17]:

```
# Business rule check: clicks should not exceed impressions
df[df["clicks"] > df["impressions"]]
```

Out[17]:

	user_id	event_date	platform	country	creative	impressions	clicks	in
21200	21201	NaT	Android	New Zealand	NaN	10	14	
21201	21202	NaT	NaN	Norway	video_ugc_01	24	27	
21202	21203	NaT	NaN	United Kingdom	video_ugc_01	14	19	
21203	21204	NaT	ios	NaN	video_ugc_01	24	28	
21204	21205	NaT	UK_OS	NaN	Static_Banner_02	9	13	
...	...	...	...	...	...	...	...	
24195	24196	2025-05-22 15:20:00	ios	USA	Static_Banner_05	12	18	
24196	24197	2025-05-18 16:56:00	NaN	USA	Video_UGC_01	19	20	
24197	24198	2025-05-20 05:28:00	ios	NaN	Animated_04	24	30	
24198	24199	2025-05-19 11:22:00	Android	Other	Video_UGC_01	9	15	
24199	24200	2025-05-23 02:47:00	UK_OS	NaN	Static_Banner_05	10	13	

3000 rows × 11 columns

Rows where clicks exceeded impressions were treated as invalid ad interaction records and removed, because such events are impossible and could not be reliably

reconstructed.

In [18]:
```
df = df[df["clicks"] <= df["impressions"]] # Remove impossible ad interaction ro
```

In [19]:
```
df[df["spend_usd"] < 0] # Business rule check: spend should not be negative
# Negative spend is no longer present because these rows were already removed
# together with invalid records where clicks > impressions.
```

Out[19]:

user_id	event_date	platform	country	creative	impressions	clicks	install_flag	sub

In [20]:
```
df["platform"].value_counts(dropna=False) # Count of rows for each platform valu
```

Out[20]:
```
platform
Android     13846
iOS          7004
ios            64
android        61
 Android       61
IOS            55
 iOS           55
ANDROID        54
Name: count, dtype: int64
```

In [21]:
```
# Platform column contains inconsistent category labels and missing values.
# Categories will be standardized before handling missing data.
```

In [22]:
```
df["platform"] = df["platform"].str.strip().str.lower()
```

In [23]:
```
df["platform"] = df["platform"].replace({
    "ios": "iOS",
    "android": "Android"
}) # Standardize platform labels to consistent format
```

In [24]:
```
df["country"].value_counts(dropna=False) # Count of rows for each Country value
```

Out[24]:
```
country
Other              7743
Norway             5772
USA                3658
United Kingdom     2486
New Zealand        1191
Other                72
other                61
norway               51
Norway               49
UK                   20
usa                  18
new zealand          16
Usa                  15
 USA                 13
United Kingdom       12
New Zealand          12
 united kingdom      11
Name: count, dtype: int64
```

In [25]:
```
df["country"] = df["country"].str.strip().str.upper()
```

In [26]:
```
df["country"] = df["country"].replace({
    "UK": "United Kingdom",
    "UNITED KINGDOM": "United Kingdom",
    "OTHER": "Other",
    "NORWAY": "Norway",
    "NEW ZEALAND": "New Zealand"
}) # Standardize country names
```

In [27]:
```
df["creative"].value_counts(dropna=False) # Count of rows for each Creative
```

Out[27]:
```
creative
Video_UGC_01         6220
Video_UGC_03         5095
Animated_04          4576
Static_Banner_02     2787
Static_Banner_05     2272
Video_UGC_01           29
video_ugc_03           28
video_ugc_01           26
 ANIMATED_04           23
Video_UGC_03           20
Animated_04            19
 VIDEO_UGC_01          19
 VIDEO_UGC_03          18
animated_04            16
Static_Banner_02       12
 STATIC_BANNER_02      11
Static_Banner_05        9
static_banner_02        7
 STATIC_BANNER_05       7
static_banner_05        6
Name: count, dtype: int64
```

In [28]:
```
df["creative"] = df["creative"].str.strip().str.lower()
```

In [29]:
```
df["creative"].value_counts(dropna=False)
```

Out[29]:
```
creative
video_ugc_01        6294
video_ugc_03        5161
animated_04         4634
static_banner_02    2817
static_banner_05    2294
Name: count, dtype: int64
```

№ 4. Conclusion

The raw marketing dataset contained several data quality issues, including duplicate records, logical inconsistencies in funnel tracking, corrupted interaction rows and inconsistent categorical labels.

After profiling, cleaning and validation, the dataset became analytically suitable for the next stage of the project.

The next step will be to reconstruct campaign KPIs from the cleaned dataset and use them as the basis for further analysis and dashboard development.

In [ ]:
```
df.to_csv("marketing_cleaned_2025.csv", index=False)
```

In [ ]:
```

```